EF Core: Include() vs Projection

N+1 → Include → Projection | Concise lesson summary

Core idea

There are three approaches to retrieving related information. Choose based on what the feature needs: avoid repeated database queries, avoid loading unnecessary data, and shape results for the screen or report.

Mental anchor: Need full related entities? Consider Include(). Need selected fields or summaries? Prefer projection.

Three approaches compared

Approach	What happens	Trade-off
N+1	Load students, then query related data separately for each student.	Many database round trips; often slow as row count grows.
Include()	Eager-load students and their related enrollments.	Useful for full related entities, but may load extra data.
Projection	Select only the needed fields and computed values.	Often best for reports needing summaries or selected fields.

1. Bad: N+1 problem

```
var students = context.Students.ToList();

foreach (var student in students)
{
int count = context.Enrollments
.Count(e => e.StudentId == student.StudentId);
}
```

One query loads students; each Count() executes another query. For 100 students, this pattern can produce 101 queries.

2. Better: Include()

```
var students = context.Students
.Include(s => s.Enrollments)
.ToList();

foreach (var student in students)
{
Console.WriteLine(
$"{student.FirstName} {student.LastName} - " +
$"{student.Enrollments.Count}");
}
```

Use Include() when you need the related enrollment entities. A simple one-to-many relationship commonly uses a LEFT JOIN, but split-query settings and query shape can change the SQL/query count.

3. Often best for reports: Projection

```
var result = context.Students
.Select(s => new
{
s.FirstName,
s.LastName,
EnrollmentsCount = s.Enrollments.Count()
})
.ToList();
```

Only the requested fields and enrollment count are returned. EF Core can translate this pattern so the database calculates the count.

Typical SQL shapes

```
-- N+1: repeated query per student
SELECT COUNT(*) FROM Enrollments
WHERE StudentId = @StudentId;

-- Include: student and enrollment columns
SELECT ... FROM Students AS s
LEFT JOIN Enrollments AS e
ON s.StudentId = e.StudentId;

-- Projection: selected fields + count
SELECT s.FirstName, s.LastName, ...
FROM Students AS s;
```

These are illustrative shapes; exact SQL depends on EF Core version, provider, and query configuration.

Decision rule

Use Include() when you need related entity objects or collections for further work. **Use projection** when the UI needs specific fields, counts, sums, averages, or other summaries. Avoid loading full related entities just to display a small report.

Inspecting queries

ToQueryString() previews the SQL shape of an IQueryable query without executing it. Runtime logging through LogTo() shows the SQL actually executed. Use logging to confirm query count and check for N+1 behavior.

Professional checklist

- Avoid database calls inside loops when a set-based query can do the work.
- Include() is useful, but not automatically the most efficient option.
- Project only the columns and calculations the feature needs.
- Check generated SQL and runtime logs rather than assuming every query is one SQL statement.

Remember: Include() helps load related entities; projection often improves efficiency further by returning only the data you need.